value to keys that do not exist.

```
from collections import defaultdict
d = defaultdict(list)

d['fruits'].append('apple')
d['vegetables'].append('carrot')
print(d)
print(d['juices'])
```

Output

```
defaultdict(<class 'list'>, {'fruits': ['apple'], 'vegetables': ['carrot']})
[]
```

Deque:

It stands for 'Double-ended Queue', which is a type of data structure that allows to add/remove elements from both ends efficiently.

- Allows fast insertion and deletion from both ends
- Works as both queue (FIFO) and stack (LIFO)

Operations -

1. **append()** —> adds an element to the right end of the queue.
2. **appendleft()** —> adds an element to the left end of the queue.
3. **extend()** —> adds multiple elements to the right end of the queue.
4. **extendleft()** —> adds multiple elements to the left end of the queue

---

Day 4

Python functions:

They are reusable blocks of code used to perform a specific task. They help organise programs and execute the same logic whenever needed by calling the function.

A function can be defined using the "**def()**".

```
def fun():
    print("Welcome to GFG")
```

After creating a function, it can be called using the name of the function followed by parentheses.

```
def fun():
    print("Welcome to GFG")

fun()
```

Output

```
Welcome to GFG
```

Function parameters:

A parameter is the variable defined within the parentheses when we declare a function.

```
# Here a,b are the parameters
def sum(a,b):
  print(a+b)

sum(1,2)
```

Output

```
3
```

Function arguments:

An argument is a value that is passed to a function when it is called.

```
def sum(a,b):
  print(a+b)

# Here the values 1,2 are arguments
sum(1,2)
```

Output

```
3
```

Positional arguments:
These are needed to be included in proper order; first argument is always listed first when the function is called, second argument needs to be called second and so on.

```
def fun(s1,s2):
  print(s1+s2)

fun("Geeks","forGeeks")
```

Output

```
GeeksforGeeks
```

Keyword arguments:
It is an argument passed to a function which is preceded by a keyword and "**=**" sign. The **order** of keyword argument **does not matter** because values are explicitly assigned.

```
def fun(s1, s2):
    print(s1 + s2)


# Here we are explicitly assigning the values
fun(s1="Geeks", s2="forGeeks")
```

Output

```
GeeksforGeeks
```

***args:**

It allows a function to accept any number of positional arguments. This is useful when the number of arguments is not known beforehand.

```
def multiply(*args):
    result = 1
    for num in args:
        result *= num
    return result

print(multiply(2, 3, 4))
```

Output

```
24
```

****kwargs:**

It allows a function to accept any number of keyword arguments. All arguments are collected into a dictionary, where the argument names becomes the keys and their corresponding values become dictionary values.

```python
def fun(**kwargs):
    for k, val in kwargs.items():
        print(k, "=", val)

fun(s1='Python', s2='is', s3='Awesome')
```

Output

```
s1 = Python
s2 = is
s3 = Awesome
```

Return values:

The "***return()***" statement is used inside a function to send a value back to the place where the function was called.

- Once "***return()***" is executed, the function stops running; any code written after it is ignored.
- If ***no value*** is ***returned*** python automatically returns "***None***".

```python
def square(n):
    return n * n

res = square(4)
print(res)
```

Output

```
16
```

Returning multiple values:

A function can return multiple values at a time. These values are automatically grouped into a tuple and can be unpacked into separate variables.

```
def get_vals():
    x = 10
    y = 20
    return x, y

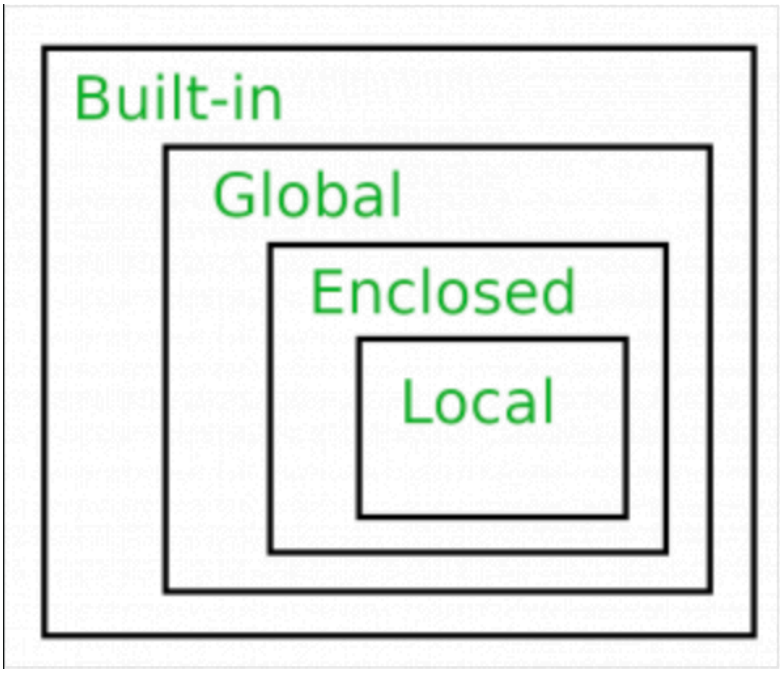
a, b = get_vals()
print(a)
print(b)
```

Output

```
10
20
```

Scope resolution: LEGB (Local, Enclosed, Global, Built-in)

It determines how variable names are searched and resolved during program execution.



Local scope:

It refers to variables defined in the current function. A function always first looks up a variable name in its local scope, if not found there, then outer scopes are checked.

```
message = "Global"
def display():
    message = "Local"
    print(message)

display()
```

Output

```
Local
```

Enclosed scope:
It refers to the scope of an enclosing function in nested functions. If a variable is not found in the local scope, python searches the enclosing function before moving to the global scope.

```
def outer():
    message = "Enclosed"

    def inner():
        print(message)

    inner()

outer()
```

Output

```
Enclosed
```

Global scope:
It contains variables defined outside all functions. These variables are accessible throughout the module and can be read inside functions unless a local variable with the same name exists.

```
message = "Global"

def display():
    print(message)

display()
```

```
Output

Global
```

Built-in scope:
It contains names that are predefined by python, such as functions, exceptions, and constants. If a name is not found in either of the other scopes, python searches the built-in scope.

```
numbers = [4, 8, 2]

print(len(numbers))
print(max(numbers))
```

```
Output

3
8
```

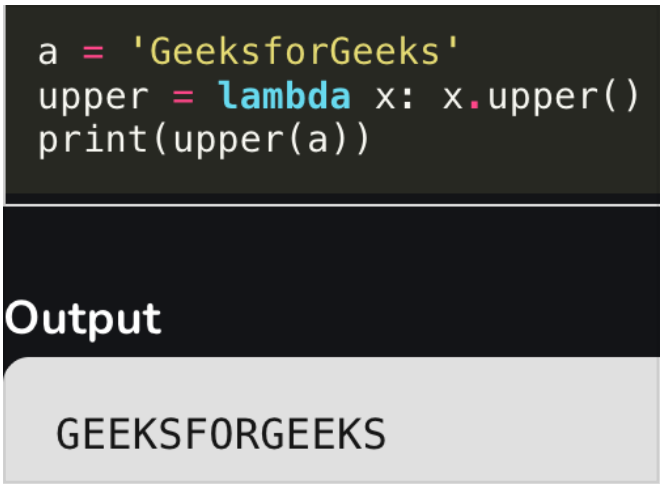
LEGB rule:
Python uses the LEGB rule to determine where to look for a variable/function name when it is referenced. It searches namespaces in a fixed order and stops as soon as it finds a matching name.

- **Local (L)**: names defined inside the current function.
- **Enclosed (E)**: names defined in the enclosing function of a nested function.
- **Global (G)**: names defined at the module (file) level.
- **Built-in (B)**: names provided by Python's built-in namespace, such as print(), len(), max().

Lambda functions/expressions:

They are small anonymous functions, meaning they do not have a defined name. These are small, short-lived functions used to pass simple logic to another function.

- Contains only one expression
- Result of that expression is returned automatically (**no *'return'* keyword needed**)



Functions as first-class objects:

In Python, functions are treated as first-class objects. This means they can just be used as numbers, strings or any other variable.

Characteristics:

- Assign functions to variables
- Passing functions as arguments
- Returning functions from other functions
- Storing functions is data structures

```python
def msg(name):
    return f"Hello, {name}!"

# Assigning the function to a variable
f = msg

# Calling the function using the variable
print(f("Emma"))
```

Output

```
Hello, Emma!
```

```python
def msg(name):
    return f"Hello, {name}!"

def fun1(fun2, name):
    return fun2(name)

# Passing the msg function as an argument
print(fun1(msg, "Alex"))
```

Output

```
Hello, Alex!
```

```python
def fun1(msg):
    def fun2():
        return f"Message: {msg}"
    return fun2

# Getting the inner function
func = fun1("Hello, World!")
print(func())
```

Output

```
Message: Hello, World!
```

```python
def add(x, y):
    return x + y

def subtract(x, y):
    return x - y

# Storing functions in a dictionary
d = {
    "add": add,
    "subtract": subtract
}

# Calling functions from the dictionary
print(d["add"](5, 3))
print(d["subtract"](5, 3))
```

Output

```
8
2
```

Python closures:

They allow a function to remember values from the environment in which it was created even if that environment no longer exists. Closures are used in functional programming, event handling and callback functions where state is needed to be retained without using global variables.

How closure is formed -

- A function is defined inside another function (nested function)
- The inner function references variables from the outer function
- The outer function returns the inner function

```python
def outer_function(x):
    # Outer function: takes 'x' and defines inner_function
    def inner_function(y):
        return x + y  # 'x' is remembered from outer_function
    return inner_function  # Returns inner function (closure)

# Create a closure with x = 10
closure = outer_function(10)

# Call the closure with different values of 'y'
print(closure(5))
print(closure(20))
```

Output

```
15
30
```

Higher Order Functions (HOF):

A HOF is a function that works with other functions. It can accept a function as an argument, return a function or do both.

```python
def process(func):
    return func("python")

def to_upper(text):
    return text.upper()

print(process(to_upper))
```

Output

```
PYTHON
```

- process() is a HOF as it accepts another function as an argument.
- to_upper() takes a string and converts it to uppercase
- process(to_upper) passes “python” to to_upper() and returns “PYTHON”

Recursion:

It is a programming technique where a function calls itself either directly or indirectly to solve a problem.

Uses -

- Breaking problems into smaller subproblems
- Mathematical calculations like factorial and fibonacci
- Tree and graph traversal and Divide-and-Conquer algorithms

```python
def factorial(n):
    if n == 0:  # Base case
        return 1
    else:       # Recursive case
        return n * factorial(n - 1)

print(factorial(5))
```

Output

```
120
```

```python
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)

print(fibonacci(10))
```

Output

```
55
```

DocStrings (Documentation Strings):

They are special strings used to document python code. They provide a description of what a module, class, function, method does.

- Declared using ‘’’ or “””.
- Written just below the function/class/module
- Unlike comments (‘#’), they can be accessed at runtime

```python
def my_function():
    """Demonstrates triple quotes docstring and does nothing."""
    return None

print("Using __doc__:")
print(my_function.__doc__)

print("Using help():")
help(my_function)
```

Output

```
Using __doc__:
Demonstrates triple quotes docstring and does nothing.
Using help():
Help on function my_function in module __main__:

my_function()
    Demonstrates triple quotes docstring and does no...
```

Explanation:

- **"""Demonstrates triple quotes docstring and does nothing.""":** Declares the docstring for my_function.
- **my_function.__doc__:** Directly prints the docstring text.
- **help(my_function):** Displays docstring in a structured help format.